

LAPORAN PRATIKUM PBO

PEKAN 3



Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Bapak Nurfiah

Asisten Praktikum: Uda Dawy Syauqi

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

A. Tahapan Pelaksanaan Praktikum

1. Class transaksi.java

a. Kode program

```
package Pekan3;

public class Transaksi {
    //1. mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    //Constructor
    public Transaksi (String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    //2. hanya menyediakan getter (Read-only)
    public String getIdTransaksi () {return idTransaksi;}
    public String getJenis () { return jenis;}
    public double getNominal () {return nominal;}

    public void cetakDetail () {
        System.out.println ("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal:
Rp" + nominal);
    }
}
```

b. Penjelasan

PENJELASAN KODE PROGRAM: Kelas Transaksi.Java

1. GAMBARAN UMUM

Kode ini adalah kelas Transaksi yang merepresentasikan sebuah transaksi (misalnya setor, tarik, atau transfer) dengan tiga data: ID, jenis, dan nominal. Kelas ini menerapkan konsep ENKAPSULASI, yaitu menyembunyikan data internal dan hanya membuka akses lewat method tertentu.

2. DEKLARASI PACKAGE

package Pekan3;

- Menyatakan bahwa kelas ini berada di package bernama "Pekan3".

- Package berfungsi untuk mengelompokkan kelas agar file lebih terorganisir dan menghindari bentrok nama.

3. DEKLARASI KELAS

```
public class Transaksi
```

- "public" berarti kelas ini dapat diakses dari package lain.
- Nama kelas harus sama dengan nama file: Transaksi.java.

4. ATRIBUT (PRIVATE)

```
private String idTransaksi;  
private String jenis;  
private double nominal;
```

- idTransaksi : menyimpan identitas unik transaksi.
- jenis : menyimpan jenis transaksi (contoh: "Setor", "Tarik").
- nominal : menyimpan jumlah uang transaksi (tipe double karena bisa berupa bilangan desimal).
- Modifier "private" membuat atribut hanya bisa diakses dari dalam kelas Transaksi. Kelas lain tidak bisa mengubah atau membaca langsung, misalnya t.nominal = 100 akan menyebabkan error. Inilah inti enkapsulasi (data hiding).

5. CONSTRUCTOR

```
public Transaksi (String id, String jenis, double nominal) {  
    this.idTransaksi = id;  
    this.jenis = jenis;  
    this.nominal = nominal;  
}
```

- Constructor dipanggil saat objek dibuat dengan "new", contoh:
Transaksi t = new Transaksi("TRX001", "Setor", 500000);
- Fungsinya mengisi nilai awal atribut.
- Kata kunci "this" merujuk ke atribut milik objek itu sendiri, untuk membedakannya dari parameter yang namanya sama (jenis dan nominal).
- Parameter "id" diisikan ke atribut "idTransaksi".

6. GETTER SAJA (READ-ONLY)

```
public String getIdTransaksi () { return idTransaksi; }  
public String getJenis () { return jenis; }  
public double getNominal () { return nominal; }
```

- Getter adalah method untuk MEMBACA nilai atribut private dari luar kelas.
- Kelas ini sengaja TIDAK menyediakan setter, sehingga setelah objek dibuat, nilainya tidak bisa diubah (read-only / immutable).
- Manfaatnya: data transaksi terlindungi dari perubahan yang tidak disengaja, cocok karena catatan transaksi seharusnya tidak boleh diubah setelah tercatat.

7. METHOD cetakDetail()

```
public void cetakDetail () {  
    System.out.println ("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" +  
nominal);
```

```
}
```

- Method bertipe void (tidak mengembalikan nilai) yang menampilkan detail transaksi ke layar.

- Operator "+" digunakan untuk menggabungkan teks (string) dengan nilai atribut.

- Contoh keluaran:

ID: TRX001 | Jenis: Setor | Nominal: Rp500000.0

8. CONTOH PENGGUNAAN

```
Transaksi t = new Transaksi("TRX001", "Setor", 500000);
```

```
t.cetakDetail();           // menampilkan detail
```

```
System.out.println(t.getNominal()); // membaca nominal: 500000.0
```

```
// t.nominal = 100;         // ERROR: atribut private
```

```
// t.setNominal(100);       // ERROR: setter tidak ada
```

9. KESIMPULAN

- Atribut private -> data terlindungi dari akses langsung.

- Constructor -> mengisi data saat objek dibuat.

- Hanya getter -> data bersifat read-only setelah dibuat.

- cetakDetail() -> menampilkan informasi transaksi.

Kelas ini adalah contoh sederhana penerapan enkapsulasi dalam Pemrograman Berorientasi Objek (PBO).

10. CATATAN TAMBAHAN (PENGEMBANGAN)

- Nominal bertipe double bisa menimbulkan ketidaktepatan pada perhitungan uang; pada aplikasi nyata biasanya digunakan BigDecimal.

- Constructor bisa ditambah validasi, misalnya nominal tidak boleh negatif atau nol.

2. Class Rekening.java

a. Kode program

```
package Pekan3;
import java.util.ArrayList;
public class Rekening {
    //1. mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; //Data Sensitif

    private ArrayList<Transaksi> riwayatTransaksi;

    //2. modifikasi constructor untuk menerima PIN awal
    public Rekening (String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;
        this.pin = pinAwal;

        // validasi PIN di dalam Constructor
        if (pinAwal.length () == 6 ) {
            this.pin = pinAwal;
        } else {
            System.out.println ("Peringatan: PIN tidak memenuhi syarat keamanan.
Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList <> ();
        System.out.println ("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
    }

    //3. Getter untuk atribut yang diizinkan dibaca public
    public String getNomorRekening() {return nomorRekening; }
    public String getNamaPemilik () {return namaPemilik;}

    //4. Method otentikasi internal validasi enkapsulasi
    public boolean otentikasi (String inputPin) {
        return this.pin.equals (inputPin);
    }

    // METHOD VALIDASI ATURAN PIN

    private boolean pin (String pin) {
```

```

// Cek 1: angka berulang semua, misal 111111, 222222
boolean semuaSama = true;
for (int i = 1; i < pin.length(); i++) {
    if (pin.charAt(i) != pin.charAt(0)) {
        semuaSama = false;
        break;
    }
}
if (semuaSama) {
    return false;
}

// Cek 2: angka berurutan naik, misal 123456
boolean urutNaik = true;
for (int i = 1; i < pin.length(); i++) {
    if (pin.charAt(i) != pin.charAt(i - 1) + 1) {
        urutNaik = false;
        break;
    }
}
if (urutNaik) {
    return false;
}

// Cek 3: angka berurutan turun, misal 654321
boolean urutTurun = true;
for (int i = 1; i < pin.length(); i++) {
    if (pin.charAt(i) != pin.charAt(i - 1) - 1) {
        urutTurun = false;
        break;
    }
}
if (urutTurun) {
    return false;
}

return true;
}

```

```

// CONSTRUCTOR

```

```

public Rekening(String nomor, String nama, double saldoAwal) {

    this.nomorRekening = nomor;
}

```

```

        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        //Wajib menginisialisasi ArrayList di dalam constructor agar tidak
        NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println(
            "Rekening atas nama " + namaPemilik
            + " berhasil dibuat dengan saldo Rp" + saldo
        );
    }

    // METHOD SETOR TUNAI

    public void setorTunai(double nominal) {

        if (nominal > 0) {

            saldo += nominal;
            //Merekam riwayat (pembuatan objek transaksi di dalam method)
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println(
                "Setor tunai Rp" + nominal
                + " berhasil. Saldo saat ini: Rp" + saldo
            );

        } else {

            System.out.println(
                "Gagal: Nominal setor harus lebih dari 0!"
            );
        }
    }

    // METHOD TARIK TUNAI (SUDAH DIPERBAIKI)

    public void tarikTunai(double nominal) {

```

```

// Minimal penarikan Rp10.000
if (nominal < 10000) {

    System.out.println(
        "Transaksi Gagal: Minimal nominal penarikan Rp10.000."
    );

    // Mengecek apakah saldo mencukupi
} else if (nominal > saldo) {

    System.out.println(
        "Transaksi Gagal: Saldo tidak mencukupi. "
        + "Saldo Anda: Rp" + saldo
    );

    // Jika semua syarat terpenuhi -> penarikan berhasil
} else {

    // Mengurangi saldo
    saldo -= nominal;

    // === Merekam riwayat transaksi penarikan ===
    String idTrx = "TRX-T-" + System.currentTimeMillis();
    Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
    riwayatTransaksi.add(trxBaru);
    // =====

    System.out.println(
        "Tarik tunai Rp" + nominal
        + " berhasil. Saldo saat ini: Rp" + saldo
    );
}
}

// METHOD CEK INFORMASI

public void cekInformasi() {

    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}

```



```

// METHOD CEK TRANSAKSI TERBARU

public void cekTransaksiTerbaru() {

    if (riwayatTransaksi.size() > 3) {
        Transaksi transaksiTerbaru =
            riwayatTransaksi.get(riwayatTransaksi.size() - 1);
        System.out.println("--- TRANSAKSI TERBARU ---");
        System.out.println("ID : " + transaksiTerbaru.getIdTransaksi());
        System.out.println("Jenis : " + transaksiTerbaru.getJenis());
        System.out.println("Nominal : Rp" + transaksiTerbaru.getNominal());
        System.out.println("-----");
    } else {
        System.out.println(
            "Transaksi belum lebih dari 3");
    }
}

// METHOD CETAK MUTASI

public void cetakMutasi() {

    System.out.println("--- MUTASI REKENING (" + nomorRekening + ") ---");

    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi trx: riwayatTransaksi) {
            trx.cetakDetail();
        }
    }

    System.out.println("-----");
}

} // <-- penutup class Rekening

```

b. Penjelasan

PENJELASAN KODE PROGRAM: Kelas Rekening.Java

1. GAMBARAN UMUM

Kelas Rekening merepresentasikan rekening bank sederhana. Kelas ini menyimpan data nasabah (nomor rekening, nama, saldo, PIN) dan riwayat transaksi, serta menyediakan operasi setor tunai, tarik tunai, cek informasi, dan cetak mutasi. Kelas ini

menerapkan ENKAPSULASI: semua data disembunyikan (private) dan hanya bisa diakses lewat method yang disediakan. Kelas ini juga berelasi dengan kelas Transaksi (hubungan "has-a": Rekening memiliki banyak Transaksi).

2. PACKAGE DAN IMPORT

```
package Pekan3;
```

```
import java.util.ArrayList;
```

- package Pekan3 : kelas berada satu package dengan kelas Transaksi, sehingga bisa saling dipakai tanpa import.

- import ArrayList : dibutuhkan agar bisa memakai ArrayList (list dinamis) untuk menyimpan riwayat transaksi.

3. ATRIBUT (SEMUA PRIVATE)

```
private String nomorRekening;
```

```
private String namaPemilik;
```

```
private double saldo;
```

```
private String pin; // data sensitif
```

```
private ArrayList<Transaksi> riwayatTransaksi;
```

- Modifier private membuat atribut tidak bisa diakses langsung dari luar kelas (misal r.saldo = 999999 akan error). Ini mencegah saldo dan PIN diubah sembarangan.

- ArrayList<Transaksi> menyimpan objek Transaksi. Setiap setor/tarik akan ditambahkan ke list ini sebagai catatan riwayat.

4. CONSTRUCTOR 1 (DENGAN PIN)

```
public Rekening(String nomor, String nama, double saldoAwal, String pinAwal)
```

- Mengisi nomorRekening, namaPemilik, saldo, dan pin.

- Ada validasi PIN: jika panjang pinAwal tepat 6 karakter, PIN dipakai. Jika tidak, tampil peringatan dan PIN diganti default "123456".

- riwayatTransaksi diinisialisasi dengan new ArrayList<>() agar tidak terjadi NullPointerException saat menambah transaksi.

- Di akhir mencetak pesan bahwa rekening berhasil dibuat.

- Catatan: baris this.pin = pinAwal; muncul dua kali (sebelum dan sesudah validasi). Yang pertama sebenarnya tidak diperlukan karena nilai pin sudah ditentukan di blok if-else.

5. CONSTRUCTOR 2 (TANPA PIN)

```
public Rekening(String nomor, String nama, double saldoAwal)
```

- Ini adalah CONSTRUCTOR OVERLOADING: dua constructor dengan nama sama tetapi parameter berbeda. Java memilih sesuai jumlah argumen saat new Rekening(...) dipanggil.

- Versi ini tidak mengisi pin, sehingga nilainya null. Akibatnya, jika otentikasi() dipanggil pada objek ini, program akan error (NullPointerException). Sebaiknya PIN diberi nilai default atau constructor ini memanggil constructor pertama memakai this(...).

6. GETTER

```
public String getNomorRekening() { return nomorRekening; }
```

```
public String getNamaPemilik() { return namaPemilik; }
```

- Hanya nomor rekening dan nama pemilik yang boleh dibaca dari luar.
- Saldo dan PIN sengaja tidak diberi getter. Saldo hanya dilihat lewat cekInformasi(), PIN tidak boleh dibaca sama sekali.
- Tidak ada setter, sehingga data tidak bisa diubah sembarangan. Saldo hanya berubah lewat setorTunai() dan tarikTunai().

7. METHOD otentikasi(String inputPin)

```
public boolean otentikasi(String inputPin) {  
    return this.pin.equals(inputPin);  
}
```

- Membandingkan PIN yang diketik pengguna dengan PIN tersimpan, dan mengembalikan true jika sama, false jika beda.
- Ini contoh enkapsulasi: pengecekan PIN dilakukan DI DALAM kelas, jadi PIN asli tidak pernah dikeluarkan ke luar.
- Gunakan equals(), bukan ==, karena untuk String yang dibandingkan adalah isinya.

8. METHOD private boolean pin(String pin)

Method ini untuk memvalidasi kekuatan PIN. Ia mengembalikan false (PIN lemah) jika:

- Cek 1: semua digit sama, contoh 111111.
- Cek 2: digit berurutan naik, contoh 123456. Logikanya: setiap karakter harus sama dengan karakter sebelumnya + 1.
- Cek 3: digit berurutan turun, contoh 654321. Setiap karakter sama dengan karakter sebelumnya - 1.

Jika lolos ketiganya, mengembalikan true (PIN cukup aman).

Cara kerja tiap cek: variabel boolean diasumsikan true, lalu perulangan for memeriksa pasangan karakter. Begitu ada yang tidak cocok, variabel diubah ke false dan perulangan dihentikan dengan break.

CATATAN PENTING:

- Method ini TIDAK PERNAH DIPANGGIL di mana pun, jadi belum berfungsi. Validasi di constructor hanya mengecek panjang PIN (6 karakter).
- Nama method sama dengan nama atribut (pin). Java mengizinkannya, tetapi membingungkan. Sebaiknya diganti, misalnya isPinValid().
- PIN default "123456" justru akan ditolak oleh aturan Cek 2 di method ini.
- Untuk memakainya, ubah kondisi di constructor menjadi:
if (pinAwal.length() == 6 && pin(pinAwal)) { ... }
- Validasi juga belum memeriksa apakah semua karakter adalah angka.

9. METHOD setorTunai(double nominal)

- Jika nominal > 0: saldo bertambah (saldo += nominal), lalu dibuat objek Transaksi baru dan ditambahkan ke riwayatTransaksi.
- ID transaksi dibuat dari "TRX-S-" + System.currentTimeMillis() (waktu saat ini dalam milidetik) agar unik. Huruf S berarti Setor.
- Jenis transaksi dicatat sebagai "Kredit" (uang masuk).
- Jika nominal <= 0: tampil pesan gagal dan saldo tidak berubah.

10. METHOD tarikTunai(double nominal)

Menggunakan if - else if - else dengan tiga kemungkinan:

- nominal < 10000 : gagal, minimal penarikan Rp10.000.
 - nominal > saldo : gagal, saldo tidak mencukupi.
 - selain itu (valid) : saldo dikurangi (saldo -= nominal), lalu dicatat sebagai Transaksi berjenis "Debit" (uang keluar) dengan ID "TRX-T-" + waktu. Huruf T berarti Tarik.
- Urutan pengecekan penting: syarat gagal dicek dulu, transaksi berhasil ada di akhir.

11. METHOD cekInformasi()

Mencetak nomor rekening, nama pemilik, dan saldo akhir. Ini satu-satunya cara "resmi" melihat saldo dari luar kelas.

12. METHOD cekTransaksiTerbaru()

- Jika jumlah riwayat > 3, method mengambil transaksi terakhir dengan riwayatTransaksi.get(riwayatTransaksi.size() - 1) lalu mencetak ID, jenis, dan nominalnya lewat getter milik Transaksi.
- Jika tidak lebih dari 3, tampil pesan "Transaksi belum lebih dari 3".
- Indeks terakhir ArrayList adalah size() - 1 karena indeks dimulai dari 0.

13. METHOD cetakMutasi()

- Mencetak header berisi nomor rekening.
- Jika riwayat kosong (isEmpty()), tampil pesan belum ada transaksi.
- Jika ada, perulangan for-each (for (Transaksi trx : riwayatTransaksi)) memanggil trx.cetakDetail() untuk setiap transaksi.

14. CONTOH PENGGUNAAN

```
Rekening r = new Rekening("1234567890", "Budi", 100000, "482913");
r.setorTunai(50000);    // saldo 150000
r.tarikTunai(20000);    // saldo 130000
r.tarikTunai(5000);    // gagal: di bawah minimal
r.cekInformasi();
r.cetakMutasi();
System.out.println(r.otentikasi("482913")); // true
// r.saldo = 999999;    // ERROR: atribut private
```

15. KESIMPULAN

- Atribut private -> data (saldo, PIN) terlindungi.

- Constructor -> mengisi data awal, menginisialisasi ArrayList, dan memvalidasi PIN.
- Getter terbatas -> hanya data yang aman yang bisa dibaca.
- otentikasi() -> cek PIN tanpa membuka PIN.
- setor/tarik -> mengubah saldo dengan validasi dan mencatat riwayat.
- ArrayList<Transaksi> -> menyimpan riwayat sebagai kumpulan objek.

16. BUG DAN PENGEMBANGAN YANG DISARANKAN

- Method pin(String) belum dipanggil, jadi perlu dipasang di constructor.
- Constructor tanpa PIN membuat pin bernilai null, sehingga otentikasi() bisa error.
- Bila pinAwal bernilai null, pemanggilan pinAwal.length() akan error.
- setorTunai/tarikTunai belum meminta PIN, padahal otentikasi() sudah tersedia. Idealnya tarik tunai memakai parameter PIN dan dicek dulu.
- Tipe double kurang tepat untuk uang karena bisa tidak presisi. Gunakan BigDecimal pada aplikasi nyata.
- System.currentTimeMillis() bisa menghasilkan ID kembar jika dua transaksi terjadi di milidetik yang sama. Gunakan counter atau UUID.
- Kondisi size() > 3 pada cekTransaksiTerbaru() membuat transaksi terbaru tidak tampil jika riwayat hanya 1-3 transaksi.

3. Class Main.java

a. Kode

program

```
1 package Pekan3;
2 import java.util.ArrayList;
3
4 public class Main {
5
6     public static void main(String[] args) {
7
8         Scanner input = new Scanner(System.in);
9
10        // Menyimpan banyak objek Rekening
11        ArrayList<Rekening> daftarRekening = new ArrayList<>();
12
13        // Menyimpan rekening yang sedang aktif
14        Rekening akunAktif = null;
15
16        boolean isRunning = true;
17
18        System.out.println("=== SISTEM PERBANKAN MINI ===");
19
20        while (isRunning) {
21
22            System.out.println("\nMenu Utama:");
23            System.out.println("1. Buka Rekening Baru");
24            System.out.println("2. Setor Tunai");
25            System.out.println("3. Tarik Tunai");
26            System.out.println("4. Cek Informasi Rekening");
27            System.out.println("5. Ganti Akun");
28            System.out.println("6. Cetak Mutasi (Riwayat)");
29            System.out.println("0. Keluar");
30            System.out.print("Pilih menu: ");
31
32            int pilihan = input.nextInt();
33            input.nextLine();
34
35            switch (pilihan) {
36
37                // 1. BUKA REKENING BARU
38
39                case 1:
40
41                    System.out.print("Masukkan No Rekening: ");
42                    String no = input.nextLine();
43
44                    // Mengecek apakah nomor rekening sudah digunakan
45                    boolean sudahAda = false;
46
47                    for (Rekening rekening : daftarRekening) {
48                        if (rekening.getNomorRekening().equals(no)) {
```

Source Design

Writable

Smart Insert

3...0

```

36
37
38 // 1. BUKA REKENING BARU
39
40 case 1:
41
42     System.out.print("Masukkan No Rekening: ");
43     String no = input.nextLine();
44
45     // Mengecek apakah nomor rekening sudah digunakan
46     boolean sudahAda = false;
47
48     for (Rekening rekening : daftarRekening) {
49         if (rekening.getNomorRekening().equals(no)) {
50             sudahAda = true;
51         }
52     }
53
54     if (sudahAda) {
55         System.out.println("Gagal: Nomor rekening sudah digunakan!");
56         break;
57     }
58
59     System.out.print("Masukkan Nama Pemilik: ");
60     String nama = input.nextLine();
61
62     System.out.print("Masukkan Saldo Awal: ");
63     double saldo = input.nextDouble();
64     input.nextLine ();
65
66     System.out.print("Masukkan PIN (6 digit):");
67     String pin = input.nextLine();
68
69     // Membuat objek Rekening baru
70     Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
71
72     // Menambahkan objek ke ArrayList
73     daftarRekening.add(rekeningBaru);
74
75     // Meniadakan rekening baru sebagai akun aktif
76     akunAktif = rekeningBaru;
77
78     System.out.println("Rekening berhasil ditambahkan ke daftar.");
79
80     break;
81

```

```
// 2. SETOR TUNAI

case 2:

    if (akunAktif == null) {

        System.out.println(
            "Error: Mohon maaf, Anda belum memiliki rekening!"
        );

    } else {

        System.out.print("Masukkan nominal setor: ");
        double setor = input.nextDouble();

        // Memanggil method setorTunai()
        akunAktif.setorTunai(setor);

    }

    System.out.print ("Masukkan PIN: ");
    String pinInput = input.nextLine();

    if (!akunAktif.otentikasi(pinInput)) {
        System.out.println ("PIN salah! Transaksi dibatalkan.");
    }

    break;

// 3. TARIK TUNAI

case 3:

    if (akunAktif == null) {

        System.out.println(
            "Error: Mohon maaf, Anda belum memiliki rekening!"
        );

    } else {

        System.out.print("Masukkan nominal tarik: ");
        double tarik = input.nextDouble();
```



```

        akunAktif.cetakRendang(cariRk);
    }
    System.out.print ("Masukkan PIN: ");
    String Inputpin = input.nextLine();

    if (!akunAktif.otentikasi(Inputpin)) {
        System.out.println ("PIN salah! Transaksi dibatalkan.");
    }

    break;

// 4. CEK INFORMASI REKENING
case 4:

    if (akunAktif == null) {

        System.out.println(
            "Error: Anda belum membuka rekening."
        );

    } else {

        // Memanggil method cekInformasi()
        akunAktif.cekInformasi();
    }

    break;

// 5. GANTI AKUN
case 5:

    if (daftarRekening.isEmpty()) {

        System.out.println(
            "Belum ada rekening yang terdaftar."
        );

    } else {

        System.out.print("Masukkan No Rekening yang ingin digunakan:
        String nomorCari = input.nextLine();

```

```
// 5. GANTI AKUN
```

```
case 5:
```

```
    if (daftarRekening.isEmpty()) {  
        System.out.println(  
            "Belum ada rekening yang terdaftar."  
        );  
    } else {  
        System.out.print("Masukkan No Rekening yang ingin digunakan:  
String nomorCari = input.nextLine();  
  
        Rekening rekeningDitemukan = null;  
  
        // Mencari rekening berdasarkan nomor  
        for (Rekening rekening : daftarRekening) {  
            if (rekening.getNomorRekening().equals(nomorCari)) {  
                rekeningDitemukan = rekening;  
                break;  
            }  
        }  
  
        if (rekeningDitemukan != null) {  
            akunAktif = rekeningDitemukan;  
  
            System.out.println(  
                "Berhasil berganti akun."  
            );  
  
            System.out.println(  
                "Akun aktif: " + akunAktif.getNamaPemilik()  
            );  
        } else {  
            System.out.println(  
                "Gagal: Nomor rekening tidak ditemukan!"  
            );  
        }  
    }  
}
```

```

        // 6. CETAK MUTASI (RIWAYAT)

        case 6:

            if (akunAktif == null) {

                System.out.println(
                    "Error: Anda belum membuka rekening."
                );

            } else {

                // Memanggil method cetakMutasi()
                akunAktif.cetakMutasi();

            }

            break;

        // 0. KELUAR

        case 0:

            isRunning = false;

            System.out.println(
                "Terima kasih telah menggunakan Sistem Perbankan Mini."
            );

            break;

        // PILIHAN TIDAK VALID

        default:

            System.out.println(
                "Pilihan menu tidak tersedia!"
            );

        }

    }

    input.close();
}

```

b. Penjelasan

PENJELASAN KODE PROGRAM: Kelas Main (Java)

1. GAMBARAN UMUM

Main adalah kelas utama (titik masuk program) yang berisi method main(). Kelas ini menampilkan menu interaktif di konsol untuk mengelola banyak rekening bank. Main

bertugas menerima input pengguna dan memanggil method dari kelas Rekening. Seluruh logika perbankan (saldo, PIN, riwayat) ada di kelas Rekening, sedangkan Main hanya mengatur alur program.

2. PACKAGE DAN IMPORT

```
package Pekan3;  
import java.util.ArrayList;  
import java.util.Scanner;
```

- package Pekan3 : satu package dengan Rekening dan Transaksi, jadi tidak perlu import untuk keduanya.
- ArrayList : menyimpan banyak objek Rekening.
- Scanner : membaca input dari keyboard.

3. VARIABEL UTAMA DI main()

```
Scanner input = new Scanner(System.in);  
ArrayList<Rekening> daftarRekening = new ArrayList<>();  
Rekening akunAktif = null;  
boolean isRunning = true;
```

- input : objek pembaca input keyboard.
- daftarRekening : "database" sederhana berisi semua rekening yang sudah dibuka.
- akunAktif : rekening yang sedang dipakai. Awalnya null (belum ada akun).
- isRunning : penentu perulangan. Jika false, program berhenti.

4. PERULANGAN MENU (while) DAN SWITCH

```
while (isRunning) { ... }
```

- Menu ditampilkan berulang sampai pengguna memilih 0.
- int pilihan = input.nextInt(); lalu input.nextLine();
nextInt() hanya membaca angka dan meninggalkan karakter Enter (newline) di buffer. Maka nextLine() dipanggil untuk membuang sisa newline itu, agar input String berikutnya tidak terlewat.
- switch(pilihan) memilih blok case sesuai angka menu. Setiap case diakhiri break agar tidak "jatuh" ke case berikutnya. default menangani pilihan yang tidak ada.

5. CASE 1: BUKA REKENING BARU

- Meminta nomor rekening, lalu mengecek keunikan dengan for-each pada daftarRekening. Jika ada yang sama (equals), sudahAda menjadi true.
- Jika sudahAda, tampil pesan gagal dan break (keluar dari switch, kembali ke menu).
- Jika belum ada, program meminta nama, saldo awal (nextDouble, lalu nextLine untuk membuang newline), dan PIN.
- Membuat objek: new Rekening(no, nama, saldo, pin), menambahkannya ke daftarRekening, dan menjadikannya akunAktif.
- Catatan: jika PIN tidak 6 karakter, constructor Rekening otomatis memakai PIN default "123456" dan hanya menampilkan peringatan.

6. CASE 2: SETOR TUNAI

- Jika `akunAktif == null`, tampil error belum punya rekening.
- Jika ada, program meminta nominal lalu memanggil `akunAktif.setorTunai(setor)`.
- Setelah itu ada blok verifikasi PIN memakai `akunAktif.otentikasi(pinInput)`. Jika salah, tampil "PIN salah! Transaksi dibatalkan."

7. CASE 3: TARIK TUNAI

Alurnya sama dengan setor: meminta nominal, memanggil `akunAktif.tarikTunai(tarik)`, lalu meminta PIN dan memeriksanya dengan `otentikasi(Inputpin)`.

8. CASE 4: CEK INFORMASI REKENING

Jika akun aktif ada, memanggil `akunAktif.cekInformasi()` untuk menampilkan nomor rekening, nama, dan saldo.

9. CASE 5: GANTI AKUN

- Jika `daftarRekening` kosong, tampil pesan belum ada rekening.
- Jika tidak, meminta nomor rekening, lalu mencari di daftar dengan `for-each`. Jika ketemu, variabel `rekeningDitemukan` diisi dan perulangan dihentikan dengan `break`.
- Jika ditemukan, `akunAktif` diganti ke rekening tersebut dan nama pemiliknya ditampilkan (`getNamaPemilik()`). Jika tidak, tampil pesan gagal.
- Ini menunjukkan bahwa `akunAktif` hanyalah referensi ke objek yang ada di dalam daftar, sehingga mengganti akun sama dengan mengganti referensi.

10. CASE 6: CETAK MUTASI

Jika akun aktif ada, memanggil `akunAktif.cetakMutasi()` untuk menampilkan seluruh riwayat transaksi.

11. CASE 0: KELUAR

`isRunning = false`; sehingga perulangan `while` berhenti. Setelah keluar dari loop, `input.close()` menutup Scanner.

12. KONSEP OOP YANG TERLIHAT

- Objek dan kelas : banyak objek `Rekening` dibuat dari satu kelas.
- Koleksi objek : `ArrayList<Rekening>` mengelola banyak akun.
- Enkapsulasi : Main tidak menyentuh saldo atau PIN langsung, hanya lewat method (`setorTunai`, `tarikTunai`, `otentikasi`, dan lainnya).
- Referensi objek : `akunAktif` menunjuk ke objek di dalam daftar.

13. BUG YANG PERLU DIPERBAIKI

a) PIN dicek SETELAH transaksi (case 2 dan 3).

`setorTunai()` dan `tarikTunai()` sudah dijalankan sebelum PIN diperiksa. Jadi walaupun PIN salah, saldo tetap berubah. Pesan "Transaksi dibatalkan" tidak benar-benar membatalkan apa pun. PIN harus diperiksa SEBELUM transaksi.

b) Newline tersisa setelah nextDouble() (case 2 dan 3).

Setelah input.nextDouble() tidak ada input.nextLine(), sehingga input.nextLine() untuk PIN langsung membaca string kosong. Akibatnya PIN selalu dianggap salah dan pengguna tidak sempat mengetik PIN.

c) NullPointerException jika akun belum ada.

Blok PIN berada di luar if-else. Jika akunAktif masih null, akunAktif.otentikasi(...) dipanggil dan program crash.

d) Input non-angka pada menu.

Jika pengguna mengetik huruf saat nextInt(), program melempar InputMismatchException. Bisa ditangani dengan try-catch atau membaca dengan nextLine() lalu Integer.parseInt().

e) Validasi lain yang belum ada.

Saldo awal negatif masih diterima, dan ganti akun (case 5) tidak meminta PIN.

14. PERBAIKAN CASE 2 (CASE 3 SAMA POLANYA)

case 2:

```
if (akunAktif == null) {
    System.out.println("Error: Mohon maaf, Anda belum memiliki rekening!");
} else {
    System.out.print("Masukkan PIN: ");
    String pinInput = input.nextLine();

    if (akunAktif.otentikasi(pinInput)) {
        System.out.print("Masukkan nominal setor: ");
        double setor = input.nextDouble();
        input.nextLine(); // buang newline
        akunAktif.setorTunai(setor);
    } else {
        System.out.println("PIN salah! Transaksi dibatalkan.");
    }
}
break;
```

Perubahan: PIN diminta dan diperiksa lebih dulu, transaksi hanya berjalan jika PIN benar, semua kode berada di dalam blok else (aman dari null), dan nextLine() ditambahkan setelah nextDouble().

15. KESIMPULAN

Main adalah antarmuka konsol berbasis menu yang mengatur banyak objek Rekening lewat ArrayList dan akunAktif. Alurnya: tampil menu, baca pilihan, jalankan case yang sesuai, ulangi sampai memilih 0. Struktur programnya sudah baik, tetapi urutan

verifikasi PIN pada setor dan tarik masih keliru dan perlu diperbaiki seperti pada poin 14.

B. Challenge

1. Soal

• Challenge:

- ☐ aturan jika pin salah tiga kali maka tampilkan akun terblokir
- ☐ fitur ganti pin dan pastikan pin sebelumnya tidak bisa digunakan
- ☐ pin hanya boleh terdiri dari 6 digit angka, input diulang
- ☐ buat aturan baru untuk setiap pin yang digunakan tidak boleh menggunakan angka yang berulang maupun angka berurutan (increasing/decreasing)

2. Penjelasan

a. Challenge pertama

Kode program

Menambahkan di class main.java

```
19 }
20 //verifikasi PIN sebelum transaksi. ch 1
21 static boolean verifikasiPin(Scanner input, Rekening r) {
22     if (r.isTerblokir()) {
23         System.out.println("Akun TERBLOKIR. Transaksi tidak dapat dilakukan.");
24         return false;
25     }
26     System.out.print("Masukkan PIN: ");
27     String p = input.nextLine();
28     if (r.otentikasi(p)) {
29         return true;
30     }
31     if (r.isTerblokir()) {
32         System.out.println("PIN salah 3 kali. Akun TERBLOKIR.");
33     } else {
34         System.out.println("PIN salah! Sisa percobaan: " + r.getSisaPercobaan());
35     }
36     System.out.println("Transaksi dibatalkan.");
37     return false;
38 }
39
```

Menambahkan di class rekening.java

```
// menghitung salah pin + blokir. ch 1
public boolean otentikasi(String inputPin) {
    if (terblokir) {
        return false;
    }
    if (pin.equals(inputPin)) {
        gagalBerturut = 0;
        return true;
    }
    gagalBerturut++;
    if (gagalBerturut >= MAKS_GAGAL) {
        terblokir = true;
    }
    return false;
}
```


output

```
1 4. Cek Informasi Rekening
1 5. Ganti Akun
1 6. Cetak Mutasi (Riwayat)
1 7. Ganti PIN
1 0. Keluar
1 Pilih menu: 3
1 Masukkan PIN: 232345
1 PIN salah! Sisa percobaan: 2
1 Transaksi dibatalkan.
2
2 Menu Utama:
2 1. Buka Rekening Baru
2 2. Setor Tunai
2 3. Tarik Tunai
2 4. Cek Informasi Rekening
2 5. Ganti Akun
2 6. Cetak Mutasi (Riwayat)
2 7. Ganti PIN
2 0. Keluar
2 Pilih menu: 3
2 Masukkan PIN: 667867
2 PIN salah! Sisa percobaan: 1
2 Transaksi dibatalkan.
3
3 Menu Utama:
3 1. Buka Rekening Baru
3 2. Setor Tunai
3 3. Tarik Tunai
3 4. Cek Informasi Rekening
3 5. Ganti Akun
3 6. Cetak Mutasi (Riwayat)
3 7. Ganti PIN
3 0. Keluar
3 Pilih menu: 3
3 Masukkan PIN: 097656
3 PIN salah 3 kali. Akun TERBLOKIR.
3 Transaksi dibatalkan.
4
4 Menu Utama:
4 1. Buka Rekening Baru
4 2. Setor Tunai
4 3. Tarik Tunai
4 4. Cek Informasi Rekening
4 5. Ganti Akun
4 6. Cetak Mutasi (Riwayat)
4 7. Ganti PIN
4 0. Keluar
4 Pilih menu:
```

b. Challenge kedua

Kode program

Menambahkan di class main.java

```
3         break;
4
5         // 7. Ganti PIN. ch 2
6
7     case 7:
8         if (akunAktif == null) {
9
10             System.out.println("Error: Anda belum membuka rekening.");
11         } else if (akunAktif.isTerblokir()) {
12             System.out.println("Akun TERBLOKIR. Tidak dapat mengganti PIN.");
13         } else {
14             System.out.print("Masukkan PIN lama: ");
15             String pinLama = input.nextLine();
16
17             if (!akunAktif.otentikasi(pinLama)) {
18                 if (akunAktif.isTerblokir()) {
19                     System.out.println("PIN salah 3 kali. Akun TERBLOKIR.");
20                 } else {
21                     System.out.println("PIN lama salah! Sisa percobaan: " + akunAktif.getSisaPercobaan());
22                 }
23             } else {
24                 String pinBaru = bacaPinBaru(input, akunAktif);
25                 akunAktif.gantiPin(pinLama, pinBaru);
26             }
27         }
28         break;
29     }
30 }
```

Menambahkan di class rekening.java

```
    }
    // menghitung salah pin + blokir. ch 1
    public boolean otentikasi(String inputPin) {
        if (terblokir) {
            return false;
        }
        if (pin.equals(inputPin)) {
            gagalBerturut = 0;
            return true;
        }
        gagalBerturut++;
        if (gagalBerturut >= MAKS_GAGAL) {
            terblokir = true;
        }
        return false;
    }
}
```

Output

main (2) [Java Application] C:\Users\user\p2\p00n\plugins\org.eclipse.jdt.openj

```
7. Ganti PIN
0. Keluar
Pilih menu: 7
Masukkan PIN lama: 232345
Masukkan PIN (6 digit angka): 787890
PIN berhasil diganti.
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 2222
Nama Pemilik : arass
Saldo Akhir  : Rp100000.0
Status       : Aktif
-----
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 3
Masukkan PIN: 232345
PIN salah! Sisa percobaan: 2
Transaksi dibatalkan.
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:
```

c. Challenge ketiga

Kode program

Menambahkan di class main.java

```
// Input PIN sesuai aturan. ch 3
static String bacaPinBaru (Scanner input, Rekening akunAktif) {
    while (true) {
        System.out.print ("Masukkan PIN (6 digit angka): ");
        String p = input.nextLine();
        String error = Rekening.cekAturanPin (p);
        if (error == null) {
            return p;
        }
        System.out.println ("Gagal: " + error + " Silahkan ulangi.");
    }
}
```

Menambahkan di class rekening.java

```
// aturan pembentukan PIN. ch 3 dan 4
public static String cekAturanPin(String pin) {
    if (pin == null || !pin.matches("\\d{6}")) {
        return "PIN harus terdiri dari tepat 6 digit angka.";
    }

    boolean semuaSama = true;
    boolean urutNaik = true;
    boolean urutTurun = true;

    for (int i = 1; i < pin.length(); i++) {
        char sekarang = pin.charAt(i);
        char sebelum = pin.charAt(i - 1);

        if (sekarang != pin.charAt(0)) semuaSama = false;
        if (sekarang != sebelum + 1) urutNaik = false;
        if (sekarang != sebelum - 1) urutTurun = false;
    }

    if (semuaSama) return "PIN tidak boleh angka yang berulang semua (contoh 111111).";
    if (urutNaik) return "PIN tidak boleh angka berurutan naik (contoh 123456).";
    if (urutTurun) return "PIN tidak boleh angka berurutan turun (contoh 654321).";
    return null;
}
```

Output

```
terminated> Main (2) [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 1234
Masukkan Nama Pemilik: arass
Masukkan Saldo Awal: 100000
Masukkan PIN (6 digit):34345
Exception in thread "main" java.lang.IllegalArgumentException: PIN harus terdiri dari tepat 6 digit angka.
    at Pekan3.Rekening.<init>(Rekening.java:21)
    at Pekan3.Main.main(Main.java:105)
```

d. Challenge keempat

Kode program

Menambahkan di class rekening.java

```
// aturan pembentukan PIN. ch 3 dan 4
public static String cekAturanPin(String pin) {
    if (pin == null || !pin.matches("\\d{6}")) {
        return "PIN harus terdiri dari tepat 6 digit angka.";
    }

    boolean semuaSama = true;
    boolean urutNaik = true;
    boolean urutTurun = true;

    for (int i = 1; i < pin.length(); i++) {
        char sekarang = pin.charAt(i);
        char sebelum = pin.charAt(i - 1);

        if (sekarang != pin.charAt(0)) semuaSama = false;
        if (sekarang != sebelum + 1) urutNaik = false;
        if (sekarang != sebelum - 1) urutTurun = false;
    }

    if (semuaSama) return "PIN tidak boleh angka yang berulang semua (contoh 111111).";
    if (urutNaik) return "PIN tidak boleh angka berurutan naik (contoh 123456).";
    if (urutTurun) return "PIN tidak boleh angka berurutan turun (contoh 654321).";
    return null;
}
```

Output:

```
Problems Javadoc Declaration Search Console X Git Staging
<terminated> Main (2) [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\javaw.e
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 2344
Masukkan Nama Pemilik: arass
Masukkan Saldo Awal: 100000
Masukkan PIN (6 digit):123456
Exception in thread "main" java.lang.IllegalArgumentException: PIN tidak boleh angka berurutan naik (contoh 123456).
    at Pekan3.Rekening.<init>(Rekening.java:21)
    at Pekan3.Main.main(Main.java:105)
```


C. Tugas

1. Soal

1.5 Latihan /Tugas

Sistem kita sekarang menuntut **PIN** saat pembuatan akun dan memiliki fungsi **otentikasi()**. Namun belum diimplementasikan pada menu interaksi di **Main.java**.

Tugas : Mengamankan CLI dengan Otentikasi

silahkan memperbarui program **Main.java** agar mematuhi aturan keamanan baru ini:

1. **Menu 1 (Buka Rekening):** Sebelum menginstansiasi objek baru, minta *user* menginputkan PIN (String angka 6 digit).
2. **Keamanan Transaksi:** Perbarui **Menu 3 (Tarik Tunai)** dan **Menu 6 (Cetak Mutasi)**. Sebelum sistem mengeksekusi kedua *method* tersebut, sistem harus meminta *user* memasukkan PIN.
 - Panggil fungsi **akunAktif.otentikasi(pinYangDiinput)**.
 - Jika mengembalikan nilai true, jalankan transaksi.
 - Jika false, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

2. Jawaban soal pertama

Kode program

Menambahkan di class main.java (case satu yang sudah dimodifikasi)

```
case 1:

    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();

    // Mengecek apakah nomor rekening sudah digunakan
    boolean sudahAda = false;

    for (Rekening rekening : daftarRekening) {
        if (rekening.getNomorRekening().equals(no)) {
            sudahAda = true;
        }
    }

    if (sudahAda) {
        System.out.println("Gagal: Nomor rekening sudah digunakan!");
        break;
    }

    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();

    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();

    System.out.print("Masukkan PIN (6 digit):");
    String pin = bacaPinBaru(input, null);

    // Membuat objek Rekening baru
    Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);

    // Menambahkan objek ke ArrayList
    daftarRekening.add(rekeningBaru);

    // Meniadikan rekening baru sebagai akun aktif
    akunAktif = rekeningBaru;

    System.out.println("Rekening berhasil ditambahkan ke daftar.");

    break;
```

Output

```
Main (2) [Java Application] C:\Users\user1.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123
Masukkan Nama Pemilik: arass
Masukkan Saldo Awal: 100,000,000
Masukkan PIN (6 digit):Masukkan PIN (6 digit angka): 127890
Rekening atas nama arassberhasil dinuat
Rekening berhasil ditambahkan ke daftar.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 223
Masukkan Nama Pemilik: shofwaa
Masukkan Saldo Awal: 100,000,000
Masukkan PIN (6 digit):Masukkan PIN (6 digit angka): 345
Gagal: PIN harus terdiri dari tepat 6 digit angka. Silahkan ulangi.
Masukkan PIN (6 digit angka): 456
Gagal: PIN harus terdiri dari tepat 6 digit angka. Silahkan ulangi.
Masukkan PIN (6 digit angka):
```

3. Jawaban soal kedua

Kode program

Menambahkan di class main.java
(case 3 yang sudah dimodifikasi)


```

// 3. TARIK TUNAI

case 3:

    if (akunAktif == null) {

        System.out.println(
            "Error: Mohon maaf, Anda belum memiliki rekening!"
        );

    } else {
        System.out.print("Masukkan PIN: ");
        String pinYangDiInput = input.nextLine();

        // Verifikasi PIN terlebih dahulu
        if (verifikasiPin(input, akunAktif)) {

            System.out.print("Masukkan nominal tarik: ");
            double tarik = input.nextDouble();
            input.nextLine();

            // Jika PIN benar, baru melakukan penarikan
            akunAktif.tarikTunai(tarik);
        } else {

            System.out.println(
                "Akses Ditolak: PIN yang Anda masukkan salah!"
            );

        }
    }

    break;

```

(case 6 yang sudah dimodifikasi)

```

// 6. CETAK MUTASI (RIWAYAT)

case 6:

    if (akunAktif == null) {

        System.out.println(
            "Error: Anda belum membuka rekening."
        );

    } else {
        System.out.print("Masukkan PIN: ");
        String pinYangDiInput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiInput)) {

            // Memanggil method cetakMutasi()
            akunAktif.cetakMutasi();
        } else {

            System.out.println(
                "Akses Ditolak: PIN yang Anda masukkan salah!"
            );

        }
    }

    break;

```

Output

(case 3)

```
Main (2) [Java Application] C:\Users\user1.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w
Masukkan Saldo Awal: 100,000,000
Masukkan PIN (6 digit):Masukkan PIN (6 digit angka): 234512
Rekening atas nama shofwaaberhasil dinuat
Rekening berhasil ditambahkan ke daftar.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 3
Masukkan PIN: 223
Masukkan PIN: 233
PIN salah! Sisa percobaan: 2
Transaksi dibatalkan.
Akses Ditolak: PIN yang Anda masukkan salah!

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 3
Masukkan PIN: 234512
Masukkan PIN: 234512
Masukkan nominal tarik: 10,000
Tarik tunai Rp10000.0 berhasil. Saldo saat ini: Rp9.999E7

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:
```

(Case 6)

```
Akses Ditolak: PIN yang Anda masukkan salah!

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 36
Pilihan menu tidak tersedia!

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu: 6
Masukkan PIN: 234512
|-- MUTASI REKENING (123) ---
ID: TRX-T-1790785122168 | Jenis: Debit | Nominal: Rp10000.0
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:
```